

Tuning a Random Forest: GridSearchCV and RandomizedSearchCV

Contents

1. Overview	2
2. The heart disease data	3
3. A random forest out of the box	3
4. One split is not enough	4
5. Tuning one setting by hand	5
6. Grid search over a random forest	5
6.1 The grid	5
6.2 Running the search	6
6.3 The result	6
7. Randomized search	7
7.1 A bigger grid	7
7.2 A trap: bootstrap and max_samples	7
7.3 The result	8
8. Which search to use	8
9. Summary	9
10. Key terms	9

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate), 10 (Tune). See the [Course map](#).



- **Builds on:** Simple imputation (mean, median, mode, constant) ([Note 37](#)); ML pipelines ([Note 38](#)); Overfitting ([Note 91](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)).
- **Leads to:** Stacking and blending ([Note 127](#)); Random under- and oversampling ([Note 133](#)); Balanced random forest ([Note 133](#)); Optuna ([Note 134](#)).
- **Compare with:** Train-test split ([Note 13](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)); Optuna ([Note 134](#)); Bayesian optimisation ([Note 134](#)).

1. Overview

Key point: A random forest is strong out of the box; tuning its hyperparameters with a grid search or a randomized search, scored by cross-validation, can make it a little stronger.

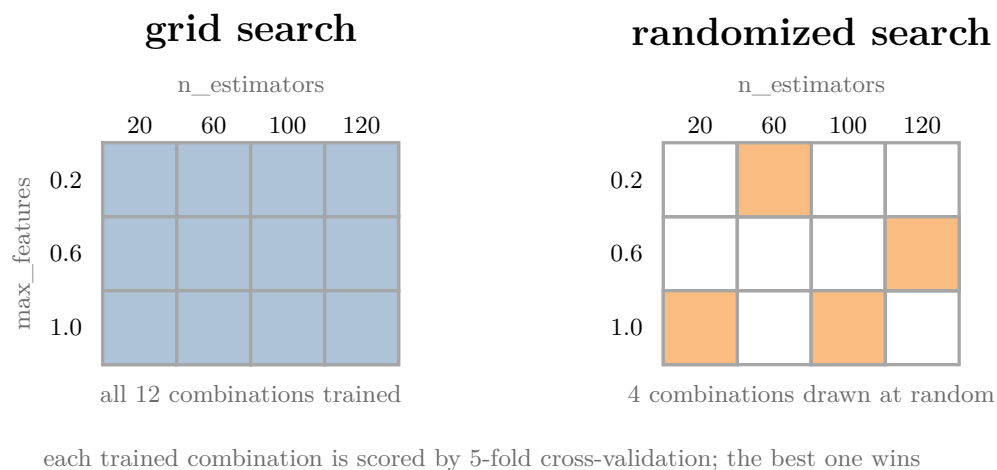


Figure 1: Grid search trains every combination of the listed values; randomized search trains a few combinations drawn at random

This Note applies a random forest to a real dataset, heart disease, and compares it with other algorithms. Then it tunes the forest's hyperparameters (the [random forest hyperparameters Note](#)) with the two search methods of Figure 1.

Both searches were introduced earlier: `GridSearchCV` in the [KNN Note](#), section 4.2, and `RandomizedSearchCV` in the [regression trees Note](#), section 7.2. Here we see what a random forest grid looks like, how long it takes, and one trap specific to forests.

The Notebook (`notebook.ipynb`) runs every step.

2. The heart disease data

Key point: 303 patients, 13 medical inputs, and a target: 1 for heart disease, 0 for none.

The data is a well-known heart disease dataset from Kaggle. Each of the 303 rows is a patient: age, sex, chest pain type (`cp`), resting blood pressure (`trestbps`), cholesterol (`chol`), maximum heart rate (`thalach`) and other medical measurements. The last column, `target`, says whether the patient has heart disease.

This is a classification problem: from the 13 inputs, predict the target. We hold out 20% for testing: 242 training rows and 61 test rows.

3. A random forest out of the box

Key point: With no tuning at all, the random forest scores 0.836 on the test split and 0.832 with cross-validation, the best cross-validated score of the four algorithms.

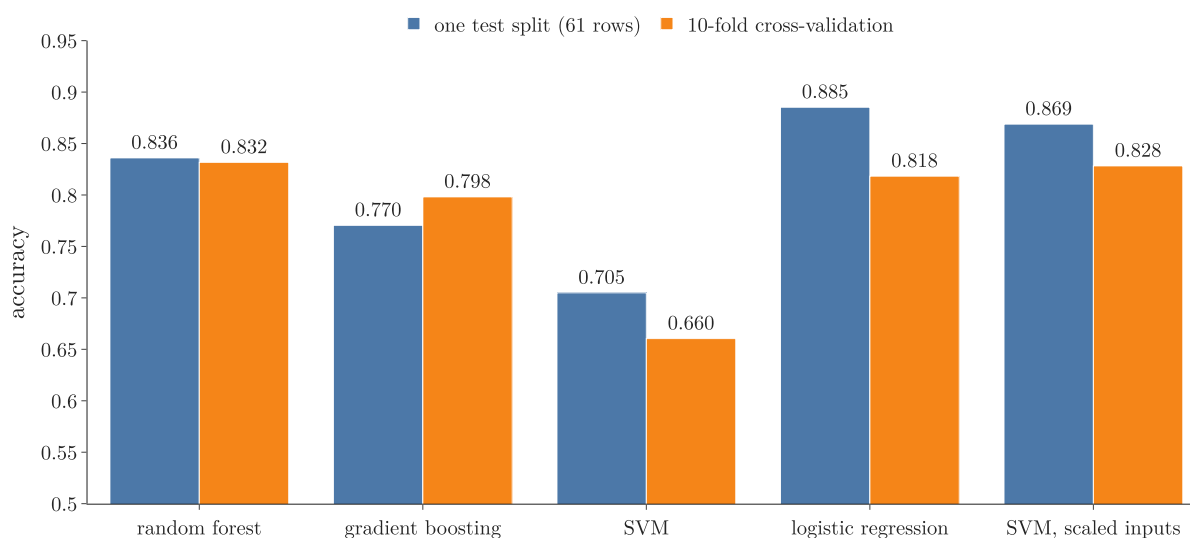


Figure 2: Accuracy of five classifiers, all with default settings, on one test split and with 10-fold cross-validation

We train four classifiers with their default settings and score each on the 61 test rows (Figure 2, blue):

Model	Test split	10-fold cross-validation
Random forest	0.836	0.832

Model	Test split	10-fold cross-validation
Gradient boosting	0.770	0.798
SVM (SVC)	0.705	0.660
Logistic regression	0.885	0.818

Gradient boosting is another tree ensemble, covered in later Notes; it is used here in the same way as any scikit-learn model. On the single test split, logistic regression wins, which suggests the pattern in this data is close to linear.

The point is not that the random forest always wins. It is that, on almost any problem, an untuned random forest lands among the top two or three algorithms.

Extra: The SVM's poor score is not a fair verdict on SVMs. An SVM measures distances, so it needs scaled inputs (the [KNN Note](#), section 7.4, shows the same for KNN), and cholesterol, in the hundreds, swamps the 0/1 columns. With a `StandardScaler` in a pipeline, the SVM scores 0.869 on the split and 0.828 with cross-validation (Figure 2, right). A random forest needs no scaling: a tree compares one column at a time with a threshold.

Logistic regression on unscaled data needs `max_iter=5000` to converge; with the default 100 iterations, scikit-learn warns that it stopped early.

4. One split is not enough

Key point: 61 test rows give a noisy score; 10-fold cross-validation trains and tests 10 times and averages, a more reliable number.

A single split of 61 rows can be lucky or unlucky. **Cross-validation** (the [pipelines Note](#), section 8) cuts the data into 10 parts, trains on 9 and tests on the tenth, 10 times, and averages the 10 scores.

With cross-validation (Figure 2, orange), logistic regression drops from 0.885 to **0.818**, while the random forest stays at **0.832**. The two are in fact close, and the forest is slightly ahead.

Python: 10-fold cross-validation.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(RandomForestClassifier(random_state=42),
                          X, y, cv=10)

scores.mean()      # 0.832
cross_val_score returns the 10 accuracies; their mean is the cross-validated score.
```

5. Tuning one setting by hand

Key point: Giving each tree 75% of the rows raises the test-split score to 0.869 and the cross-validated score to 0.838.

Changing hyperparameters can improve the forest. For example, `max_samples=0.75` gives each tree 75% of the training rows instead of a full-size bootstrap sample:

- test split: 0.836 $\rightarrow$ **0.869**;
- 10-fold cross-validation: 0.832 $\rightarrow$ **0.838**.

The cross-validated gain is small but real. The problem is scale: a random forest has about 20 hyperparameters, and guessing a good value for each by hand is hopeless. We need a systematic search.

6. Grid search over a random forest

Key point: Four hyperparameters with 4, 3, 3 and 3 values make 108 forests; 5-fold cross-validation trains each 5 times, 540 fits in all, about 14 seconds here.

6.1 The grid

Key point: List the values to try for each hyperparameter; the search trains a forest for every combination.

We choose values for the four settings that matter most:

Python: The grid for a random forest.

```
param_grid = {
    "n_estimators": [20, 60, 100, 120],    # trees
    "max_features": [0.2, 0.6, 1.0],        # columns per split
    "max_depth": [2, 8, None],              # depth of each tree
    "max_samples": [0.5, 0.75, 1.0],        # rows per tree
}
```

The grid is a **dictionary**: each key is a hyperparameter's name, exactly as the model spells it, and each value is the list of values to try.

The search tries every combination, so the number of forests is the product of the list lengths:

$$4 \times 3 \times 3 \times 3 = 108 \text{ forests}$$

With 4 hyperparameters, the combinations form a 4-dimensional table (Figure 1 shows 2 of the dimensions); every cell becomes one forest. Hence the name **grid search**.

6.2 Running the search

Key point: GridSearchCV takes the model, the grid, the number of folds, and optionally verbose and n_jobs.

Python: Grid search with 5-fold cross-validation.

```
from sklearn.model_selection import GridSearchCV

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
                        param_grid, cv=5,
                        verbose=1, n_jobs=-1)
rf_grid.fit(X_train, y_train)
# Fitting 5 folds for each of 108 candidates,
# totalling 540 fits
cv=5: every forest is trained and scored 5 times. verbose=1 prints the progress line;
n_jobs=-1 uses every CPU core, which matters when hundreds of forests are trained.
```

The search trains $108 \times 5 = 540$ forests. On this small dataset that takes about 14 seconds on 12 cores; on a large dataset it can take hours.

6.3 The result

Key point: The best forest has 20 trees of depth 8, 20% of the columns per split and 75% of the rows per tree: cross-validated accuracy 0.843.

`rf_grid.best_params_` gives the winning combination, and `rf_grid.best_score_` its mean cross-validated accuracy:

n_estimators	max_features	max_depth	max_samples	Cross-validated accuracy
20	0.2	8	0.75	0.843
20	0.2	None	0.75	0.839
20	0.6	8	1.0	0.831
20	0.2	8	1.0	0.831
60	0.6	8	1.0	0.827

The table lists the top five, from `rf_grid.cv_results_` (the [missing indicator Note](#)). The winner scores **0.869** on the 61 test rows. By default, `GridSearchCV` retrains the best combination on the whole training set, so `rf_grid` itself predicts with the best forest.

7. Randomized search

Key point: RandomizedSearchCV trains only a fixed number of random combinations (10 by default), so a much bigger grid stays affordable.

7.1 A bigger grid

Key point: Three more hyperparameters turn 108 combinations into 864; randomized search tries 10 of them, 50 fits.

We add `bootstrap`, `min_samples_split` and `min_samples_leaf`, with two values each. That gives $108 \times 2 \times 2 \times 2 = 864$ combinations, or 4,320 fits in a grid search. `RandomizedSearchCV` instead draws `n_iter` combinations at random, 10 by default: $10 \times 5 = 50$ fits (Figure 1, right).

7.2 A trap: bootstrap and max_samples

Key point: `bootstrap=False` with a `max_samples` value is not allowed, so half the random combinations fail; the fix is a list of two grids.

Putting both lists into one grid lets the search draw combinations such as `bootstrap=False`, `max_samples=0.5`. scikit-learn refuses these, because `max_samples` only applies when rows are drawn (the [random forest hyperparameters Note](#), section 3.3). The search does not stop: it records the failed combinations with the score NaN (“not a number”) and prints a warning. In our run, **5 of the 10** combinations failed, so the search really tried only 5.

The fix is to pass a **list of grids**: the search picks one of the grids at random each time, then a combination from it.

Python: Two grids, so `max_samples` only meets `bootstrap=True`.

```
from sklearn.model_selection import RandomizedSearchCV
```

```
grids = [
    dict(param_grid, bootstrap=[True],
         min_samples_split=[2, 5], min_samples_leaf=[1, 2]),
    {"n_estimators": [20, 60, 100, 120],
     "max_features": [0.2, 0.6, 1.0],
     "max_depth": [2, 8, None], "bootstrap": [False],
     "min_samples_split": [2, 5], "min_samples_leaf": [1, 2]},
]
```

```
rf_random = RandomizedSearchCV(
    RandomForestClassifier(random_state=42), grids,
    n_iter=10, cv=5, n_jobs=-1, random_state=42)
```

```
rf_random.fit(X_train, y_train)
```

`dict(param_grid, bootstrap=[True], ...)` copies `param_grid` and adds keys. The second grid has no `max_samples`, so it keeps the default, `None`. `random_state=42` fixes which combinations are drawn.

7.3 The result

Key point: In about one second, the randomized search finds a forest with cross-validated accuracy 0.826, a little below the full grid's 0.843.

The best of the 10 combinations: 20 trees, 60% of the columns, fully grown, `min_samples_split=5`, `min_samples_leaf=2`, all the rows with replacement. Its cross-validated accuracy is **0.826**, and its test accuracy 0.836.

The randomized search trained 50 forests instead of 540 (and instead of 4,320 for a grid over its larger space). It found a good forest, not the best one: that is the trade it makes.

8. Which search to use

Key point: Small data or few hyperparameters: grid search. Large data or many hyperparameters: randomized search.

- **GridSearchCV** tries everything, so it finds the best combination in the grid. Use it when the data is small or there are few values to try.
- **RandomizedSearchCV** tries a sample. Use it when the data is large or there are many hyperparameters, where a full grid would take too long. Raising `n_iter` buys a better chance of a top combination at the cost of more time.

A common approach combines them: a randomized search over a wide grid to find the promising region, then a grid search over a narrow grid around it.

9. Summary

	Grid search	Randomized search
Class	<code>GridSearchCV</code>	<code>RandomizedSearchCV</code>
Combinations tried	all (here 108)	<code>n_iter</code> (default 10)
Fits with <code>cv=5</code>	540	50
Our time	about 14 seconds	about 1 second
Best cross-validated accuracy	0.843	0.826
Use when	small data, few hyperparameters	large data, many hyperparameters

- Out of the box, the random forest had the best cross-validated score of four algorithms (0.832).
- Judge models by cross-validation, not one small test split: logistic regression fell from 0.885 to 0.818.
- A grid over 4 hyperparameters needed 108 forests and 540 fits; the best forest scored 0.843.
- `bootstrap=False` cannot be combined with `max_samples`: use a list of grids.

10. Key terms

Term	Meaning
Parameter grid	A dictionary of hyperparameter names and the values to try for each
List of grids	Several parameter grids passed together, so incompatible values never meet
<code>best_score__</code>	The best mean cross-validated score found by a search
<code>n_iter</code>	The number of random combinations <code>RandomizedSearchCV</code> tries (default 10)